Chapter 18 : Concurrency Control

Ensure the **isolation** property of transactions executed concurrently.

Database Principle and Design

Outline

- Concurrency control concepts
- Lock-based protocols
- Implementation of Locking
- Deadlock Handling
- Weak Levels of Consistency

18.1 Concurrency Control Concepts

- To ensure the **isolation** property of transactions, the **concurrency control scheme** will ensure that all possible schedules are **serializable**, **recoverable** and **cascadeless**
- The **concurrency control scheme** do not examine each schedule for serializability.
  - testing a schedule for serializability *after* it has executed is a little too late!
- Instead, it employs a **concurrency control protocol,** which imposes a discipline that avoids non-serializable schedules.

18.2 Lock-Based Protocols

Allow a transaction to access a data item only if it is currently holding a **lock** on that item.

Lock

- A **lock** is a mechanism to control concurrent access to a data item.
- To access a data item, the transaction must first lock that item.
- Transactions can lock data items in two modes :

  1. **Exclusive** *(X) mode*. Data item can be both read as well as written. **X-lock** is requested using **lock-X** operation.

  2. **Shared** *(S) mode*. Data item can only be read. **S-lock** is requested using **lock-S** operation.

- Lock requests are made to **concurrency-control scheme**. Transaction can proceed only after request is granted.
- After accessing the data item, the transaction can unlock the data item by the **unlock** operation.

Lock-compatibility

- A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions
- **Lock-compatibility matrix**

locks already held

requested lock	S	X
S	true	false
X	false	false

- The use of these two lock modes allows **multiple** transactions to **read** a data item at the same time but limits **write** access to just **one** transaction.
- If a lock cannot be granted, the requesting transaction is made to **wait** till all incompatible locks held by other transactions have been released.

Example: A Schedule With Locks

- A transaction issues the standard **read/write** operation, without explicit locking calls. Database system automatically generates the appropriate **lock** and **unlock** operations for a transaction.

B=150, A=150

T_1	T_2	concurrency-control manager
lock-X(B)		
		grant-X(B, T_1)
read(B)		
$B := B - 50$		
write(B) B=100		
unlock(B)		
	lock-S(A)	
		grant-S(A, T_2)
	read(A) A=150	
	unlock(A)	
	lock-S(B)	
		grant-S(B, T_2)
	read(B) B=100	
	unlock(B)	
	display($A + B$) 250	
lock-X(A)		
		grant-X(A, T_1)
read(A)		
$A := A + 50$		
write(A) A=200		
unlock(A)		

Note: This schedule is not serializable

Locking Protocols

Locking is **not sufficient** to guarantee serializability. Some protocols are required to restrict the set of possible schedules.

- A **locking protocol** is a set of rules followed by all transactions while requesting and releasing locks.
  - A schedule S is **legal** under a protocol if all transactions in S follow the protocol.
  - A protocol **ensures** serializability if all legal schedules under that protocol are serializable.
- Different protocols provide different tradeoffs between the amount of concurrency they allow and the amount of overhead that they incur.
- **Two-phase locking protocol** is the most frequently used protocol In practice.

Two-Phase Locking Protocol

- This protocol requires that each transaction issue lock and unlock requests in two phases:
- Phase 1: **Growing Phase**
  - Transactions may request locks
  - Transactions may not release locks
- Phase 2: **Shrinking Phase**
  - Transactions may release locks
  - Transactions may not request locks
- This protocol ensures conflict-serializable schedules. The transactions can be serialized in the order of their **lock points** (i.e., the point in the schedule where the transaction acquired its final lock).

T3	T4
	lock-S (A); read (A);
lock-X (B) read (B) B = B – 50 write (B)	
	lock-S (B); read (B); display (A + B); unlock (A); unlock (B).
lock-X (A); read (A) A = A + 50 write (A) unlock (B) unlock (A).	

Note: Although two-phase locking protocol does not avoid all dirty reads and dirty writes, it ensures that these operations will not lead to database inconsistency.

Two-Phase Locking Protocol (Cont.)

- Extensions to basic two-phase locking needed to ensure recoverability of freedom from cascading roll-back
  - **Strict two-phase locking protocol:** a transaction releases all X-locks at the end of the transaction.
  - **Rigorous two-phase locking protocol**: a transaction releases all locks at the end of the transaction..
- Both protocols ensure cascadeless.
- Most implementations of database systems use strict two-phase locking.

```
lock-S (C)
read (C)
lock-X (A);
unlock (C)
read (A)
A = A + C
write (A)
commit
unlock (A)
```

under strict two-phase locking protocol

```
lock-S (C)
read (C)
lock-X (A);
read (A)
A = A + C
write (A)
commit
unlock (C)
unlock (A)
```

under rigorous two-phase locking protocol

Lock Conversions

- Two-phase locking protocol with **lock conversions**:
  - Growing Phase:
    - can acquire a lock-S on item
    - can acquire a lock-X on item
    - can **convert** a lock-S to a lock-X (**upgrade**)
  - Shrinking Phase:
    - can release a lock-S
    - can release a lock-X
    - can convert a lock-X to a lock-S (**downgrade**)
- This protocol still ensures serializability.

Why do we need lock conversions?

Consider the schedules of T8 and T9 under rigorous two-phase locking. Without lock conversion, T8 and T9 can only run in series. With lock conversion, T8 and T9 can run more concurrently.

T9	T8
	lock-X (a_1) read (a_1) lock-S (a_2) read (a_2) . . . lock-S (a_n) read (a_n) write (a_1) commit unlock (a_1) unlock (a_2) . . . unlock (a_n)
lock-S (a_1) read (a_1) lock-S (a_2) read (a_2) display(a_1+a_2) commit unlock (a_1); unlock (a_2);	

T9	T8
lock-S (a_1) read (a_1) lock-S (a_2) read (a_2) display(a_1+a_2) commit unlock (a_1); unlock (a_2);	
	lock-X (a_1) read (a_1) lock-S (a_2) read (a_2) . . . lock-S (a_n) read (a_n) write (a_1) commit unlock (a_1) unlock (a_2) . . . unlock (a_n)

T9	T8
	lock-S (a_1) read (a_1) lock-S (a_2)
lock-S (a_1) read (a_1) lock-S (a_2) read (a_2)	
	read (a_2) . . . lock-S (a_n) read (a_n)
display(a_1+a_2) commit unlock (a_1) unlock (a_2)	
	upgrade(a_1) write (a_1) commit unlock (a_1) unlock (a_2) . . . unlock (a_n)

Starvation

- The transaction T may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted an S-lock on the same item. The transaction T may never make progress, and is said to be **starved**.
- To avoid starvation of transactions, when a transaction requests a lock on a data item Q in a particular mode M, the concurrency-control scheme grants the lock provided that:
  - There is no other transaction holding a lock on Q in a mode that is incompatible with M.
  - There is no other transaction that is waiting for a lock on Q.

18.2 Implementation of Locking

Assuming that the concurrency control scheme uses rigorous two-phase locking protocol with lock conversions and avoids starvation.

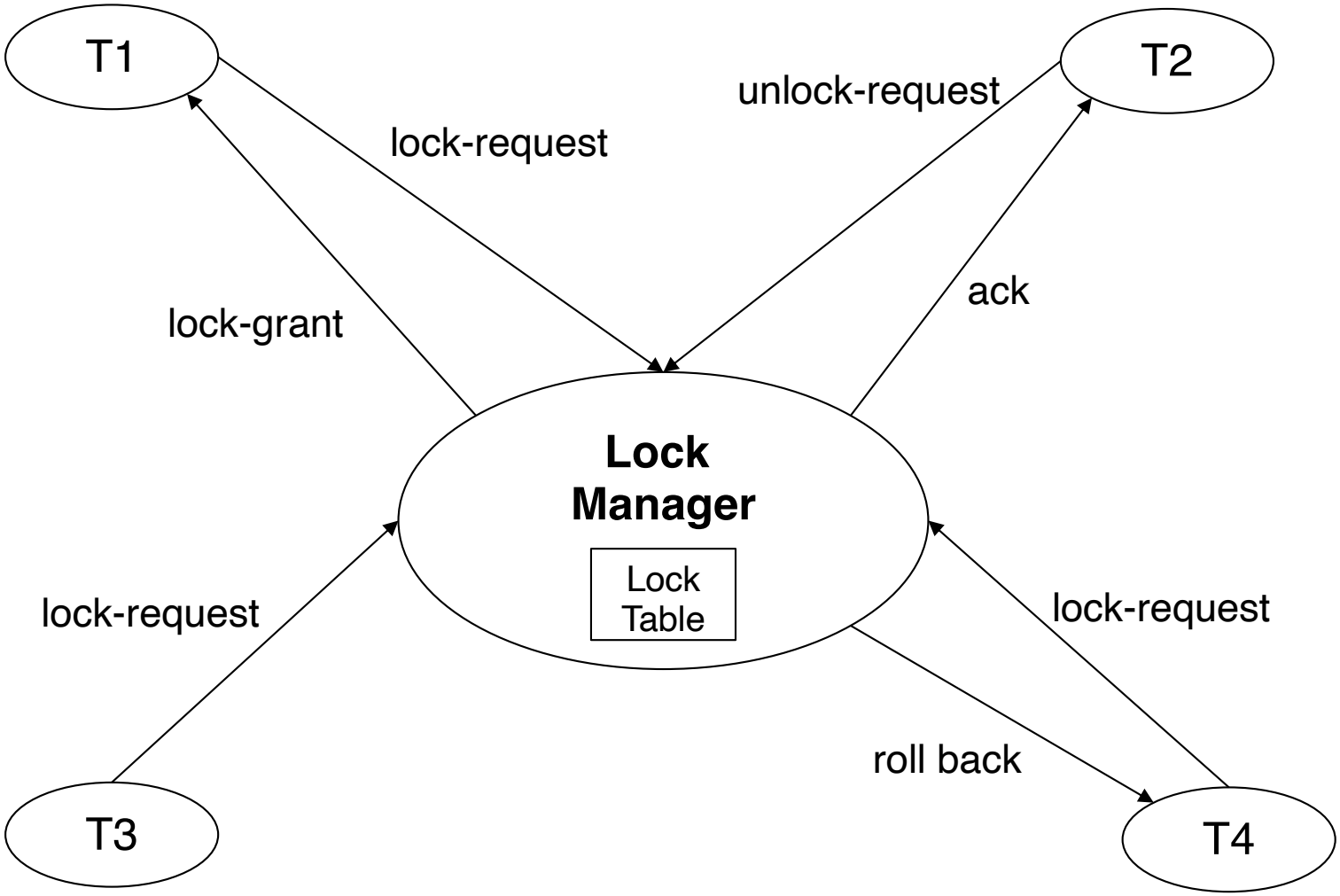
Automatic Acquisition of Locks

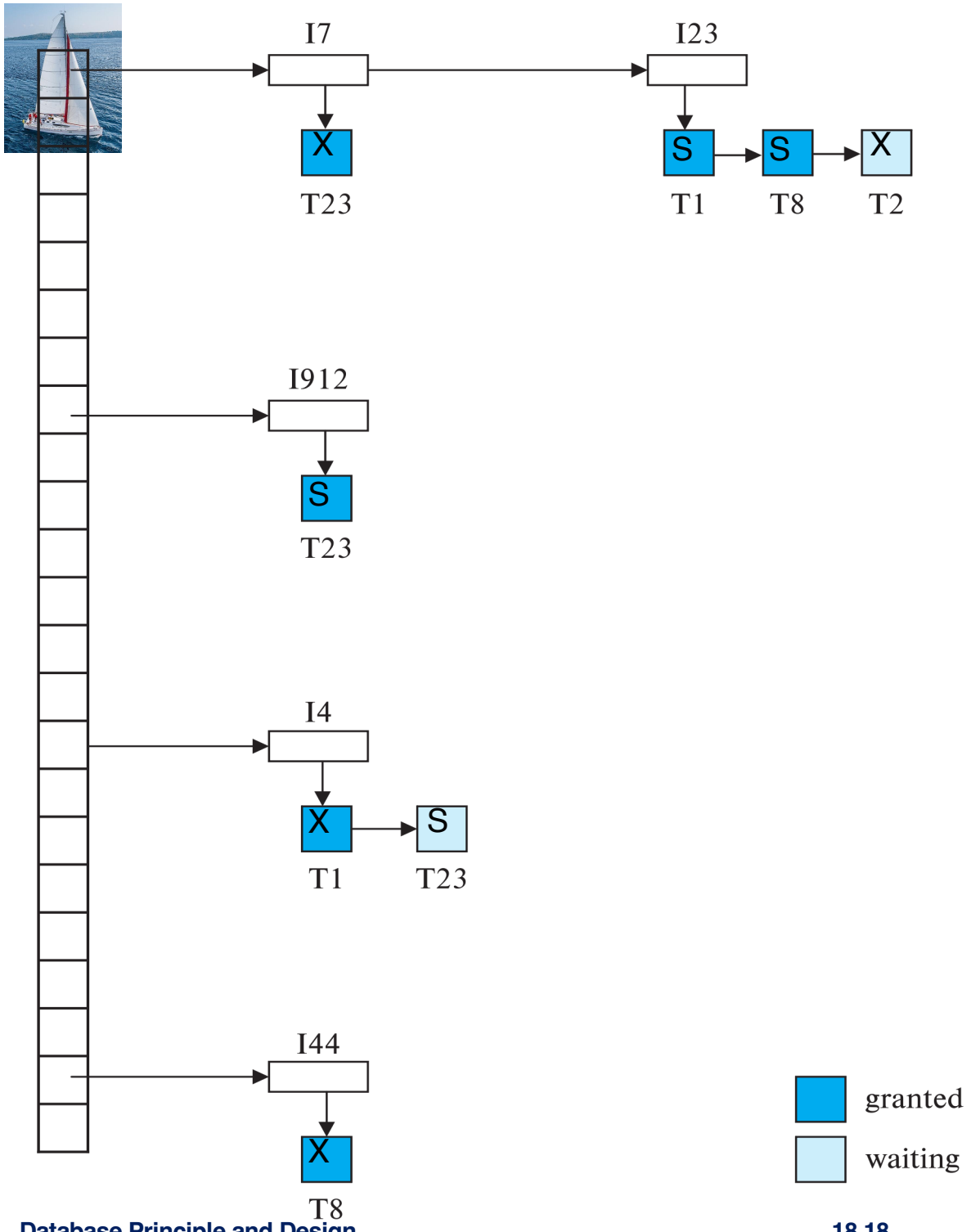
- When a transaction *T* issues a **read**(*Q*) operation, the system issues a **lock-S**(*Q*) operation before the **read**(*Q*) operation.
- When T issues a **write**(*Q*) operation, the system checks to see whether *T* already holds a shared lock on *Q*. If it does, then the system issues an **upgrade**(*Q*) operation before the **write**(*Q*) operation. Otherwise, the system issues a **lock-X**(*Q*) operation before the **write**(*Q*) operation.
- All locks obtained by a transaction are unlocked after that transaction commits or aborts.

```
lock-S (C)
read (C)
lock-S (A);
read (A)
A = A + C
upgrade (A)
write (A)
commit
unlock (C)
unlock (A)
```

Lock Manager

- Locks are managed by a process called **lock manager,** which receives lock-requests and unlock-requests from transactions and sends messages in reply.
- The lock manager replies to lock-request with lock-grant messages, or with messages asking the transaction to roll back (in case of deadlocks).
  - The requesting transaction waits until its request is answered
- Unlock-request require only an acknowledgment in response, but may result in a grant message to another waiting transaction.
- The lock manager maintains an in-memory data-structure called a **lock table** to record granted locks and pending requests


T1
T2
lock-request
unlock-request
lock-grant
ack
Lock
Manager
Lock
Table
lock-request
lock-request
roll back
T3
T4



Lock Table

- The lock table is a separate chaining hash table hashed on data items. There is a linked list of data items for each entry in the lock table.
- For each data item, the lock table maintains a linked list of records, one for each request, in the order in which the requests arrived.
- Each record notes the transaction that made the request, the locking mode of the request, and whether it was granted.

Lock Requests Processing

- When a lock request arrives, The lock manager adds a record to the end of the linked list for the data item.
- The lock manager always grants a lock request on a data item that is not currently locked. But if the transaction requests a lock on an item on which a lock is currently held, the lock manager grants the request only if it is compatible with the locks that are currently held, and all earlier requests have been granted already. Otherwise the request has to wait.

Note: Upgrade requests are omitted for simplicity

Unlock Requests Processing

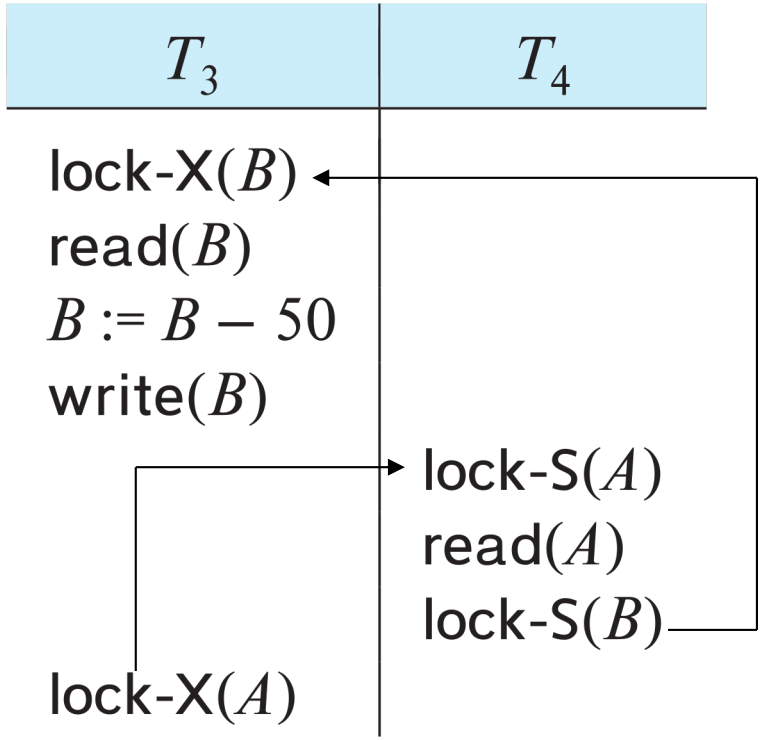
- All locks obtained by a transaction are unlocked after that transaction commits or aborts.
- When the lock manager receives an unlock request from a transaction, it deletes the record corresponding to that request. It tests the record that follows, if any, to see if that request can now be granted. If it can, the lock manager grants that request and processes the record following it, if any, similarly, and so on.

18.3 Deadlock Handling

The potential for deadlock exists in most locking protocols.

What is deadlock?

- The system is in a **deadlock** state if there exists a set of transactions such that every transaction in the set is waiting for another transaction in the set.
- Two-phase locking does not ensure freedom from deadlocks.
- Example:



- Neither T_3 nor T_4 can make progress.
- To remedy a deadlock, one of T_3 or T_4 must be rolled back and its locks released.

Deadlock Handling

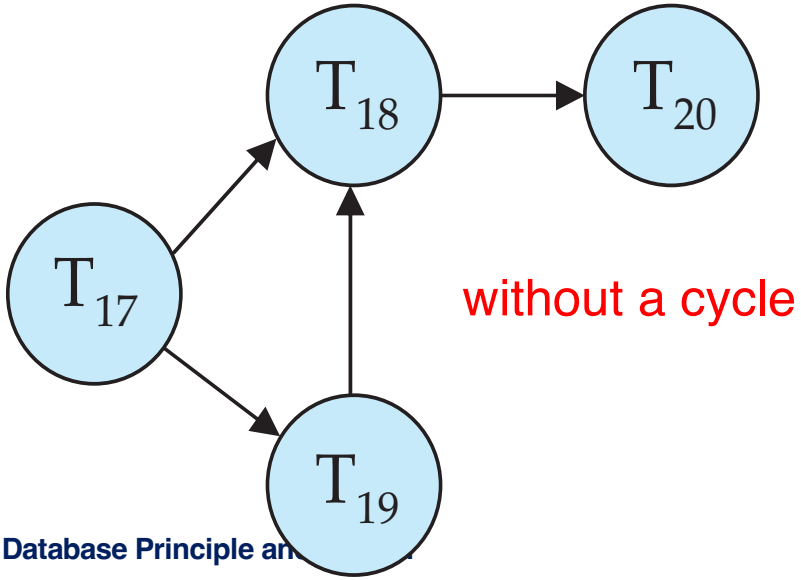
- There are two principal methods for dealing with the deadlock problem:
  - **Deadlock prevention**: ensure that the system will never enter a deadlock state.
  - **Deadlock detection and recovery**: allow the system to enter a deadlock state, and then try to recover by using a deadlock detection and deadlock recovery scheme.
- **Prevention** is commonly used if the probability that the system would enter a deadlock state is relatively high; otherwise, **detection and recovery** are more efficient.

Deadlock Prevention

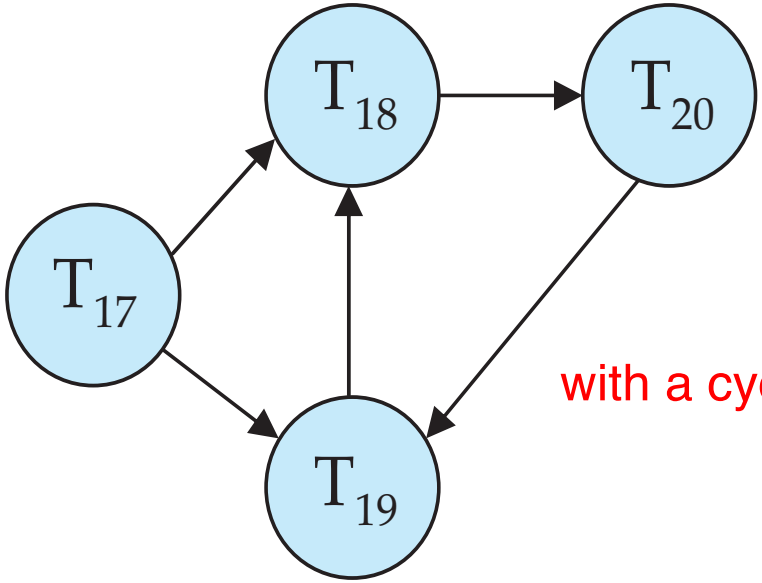
- Some prevention approaches:
  - **Pre-declaration:** require that each transaction locks all its data items before it begins execution.
    - low data utilization and low concurrency
  - **Preemptive:** older transactions never wait for younger ones, but force younger transactions to rollback instead (restarted later with its old timestamp).
    - many rollbacks; some rollbacks are unnecessary.
  - **Non-preemptive**: younger transactions never wait for older ones, they rollback instead.
  - **Timeout**: a transaction waits for a lock only for a specified amount of time. After that, the wait times out and the transaction is rolled back.
    - rollback after waiting a long time

Deadlock Detection and Recovery

- **Wait-for graph** is used for deadlock detection
  - **Vertices***:* transactions
  - **Edge** from $T_i \rightarrow T_j$: if T_i is waiting for a lock held in conflicting mode by T_j
- The system is in a deadlock state if and only if the wait-for graph has a **cycle**.
- Deadlock detection is Invoked periodically to find cycles.



without a cycle



with a cycle

Deadlock Recovery

- When deadlock is detected :
  - Select the transaction (**victim**) with the lowest cost to **rollback** to break the deadlock cycle.
  - Rollback
    - **Total rollback**: Abort the victim transaction and then restart it.
    - **Partial rollback**: Rollback the victim transaction only as far as necessary to release locks that another transaction in cycle is waiting for

End of Chapter 18